Phage
SECURITY

RugRumble

Security Review

Conducted by:

Pyro, Security Researcher
YanecaB, Security Researcher

13.11.2025

Table of Contents

Disclaimer

Audits are a time, resource, and expertise-bound effort where trained experts evaluate smart contracts using a combination of automated and manual techniques to identify as many vulnerabilities as possible. Audits can reveal the presence of vulnerabilities **but cannot guarantee their absence**.

System overview

RugRumble is a betting game on Monad where players place bets that can either win them money or lose their bet, with game results verified by admin signatures and a built-in system that shares a portion of betting fees as rewards for referrals and promotions.

Executive summary

Overview

Project Name	RugRumble
Repository	private
Commit hash	a88933e2a6d51fdf5f9872964957a845b6026fa2
Remediation	13eded644d84b78affcfc4eb1810f7c91709fa75
Methods	Manual review

Timeline

Audit kick-off	11.11.2025
End of audit	12.11.2025
Remediations start	13.11.2025
Remediations end	13.11.2025

Scope

contracts/RugRumble.sol

Issues Found

Severity	Count
High	0
Medium	0
Low	3

Findings

Low Severity

[L-01] Signatures can be replayed if the contract is upgraded

The contract uses a simple signature without EIP-712 domain separation. Signatures are generated by hashing parameters with a string prefix, where that prefix would include the chainID:

```
bytes32 messageHash = keccak256(
   abi.encode(
      string(abi.encodePacked(messagePrefix, ":createGame")),
      preliminaryGameId,
      gameSeedHash,
      algoVersion,
      gameConfig,
      msg.sender,
      msg.value,
      deadline
   )
);
```

This approach lacks proper domain separation with chain ID and contract address. If the contract is deployed on multiple chains or upgraded to a new address, signatures from one deployment could be replayed on another deployment with the same admin addresses.

Recommendation

Consider implementing EIP712 or add `address(this)` to all hashes.

Resolution: Fixed in 82e95b2

[L-02] Gas griefing via unbounded external call

The `cashOut` function uses `payable(playerAddress).call{value: payoutAmount}("")` to transfer winnings. This low-level call forwards all remaining gas to the recipient. A malicious player address with a `receive()` or `fallback()` function that is intentionally designed to be computationally expensive.

This can lead to two types of gas griefing attacks against the admin processing the transaction: - Griefing via expensive success: The malicious contract consumes a very large amount of gas but completes successfully. The `cashOut` transaction succeeds, but the admin is forced to pay an exorbitant gas fee. - Griefing via failure: The malicious contract consumes all forwarded gas, causing the `.call` to fail and the entire `cashOut` transaction to revert. The admin wastes the gas fee on a failed transaction.

Recommendation

Implement a gas limit in low-level calls.

Resolution: Fixed in 48cecfd

[L-03] `RugRumble.sol` lacks a constructor, allowing anyone to initialize it

`RugRumble` is planned to be upgradable and will be an implementation contract, where a proxy would house the storage.

```
contract RugRumble is Initializable, OwnableUpgradeable,
   ReentrancyGuardUpgradeable {
    // ...

    // ═══════════════════
    // Constructor
    // ═══════════════════

    function initialize(string memory _messagePrefix) public initializer {
```

However, even if the implementation will not be used directly, it is still better to initialize it so that a user **cannot initialize the main implementation contract**.

This concept is also explained by OpenZeppelin in Initializable.sol:

> An uninitialized contract can be taken over by an attacker. This applies to both a proxy and its implementation contract, which may impact the proxy. To prevent the implementation contract from being used, you should invoke the `_disableInitializers` function in the constructor to automatically lock it when it is deployed:

```
/// @custom:oz-upgrades-unsafe-allow constructor
constructor() {
    _disableInitializers();
}
```

Recommendation

Consider adding a constructor that will disable initializations:

```
-   // ═══════════════════
-   // Constructor
-   // ═══════════════════

+   constructor() {
+       _disableInitializers();
+   }
```

Resolution: Fixed in 9fda25a